



ALEXANDRIA UNIVERSITY

FACULTY OF ENGINEERING

ELECTRICAL POWER AND MACHINES ENGINEERING DE-
PARTMENT

Power System Protection

Microcontroller-Based Overcurrent Relay: IEC Very Inverse Time Characteristic

Prepared by:

Abd El-Rhman Muhammad Saad Muhammad

ID: 19017058

Project Repository:

[Click here to access the Arduino Code on GitHub](#)

Date: May 2026

Abstract

This report presents the design and implementation of a digital overcurrent relay that follows the IEC Very Inverse time–current characteristic. An Arduino Uno measures the load current through a ZMCT103C current transformer module, estimates its RMS value, and compares it with a pickup setting I_s . When the current exceeds I_s , the relay calculates the required operating time from the Time Multiplier Setting (TMS) and trips a relay module once the fault has lasted long enough; short disturbances that clear earlier do not cause a trip. Measurements, trip time and status are shown on an I2C LCD, two LEDs indicate the relay state, and a push button resets the relay after a trip. The prototype is rated for a 12 V AC / 1 A test bench. The report describes the hardware, the signal-processing method, the trip algorithm, the expected time–current behavior, and the limitations of the prototype together with possible improvements.

Contents

1	Introduction and Theoretical Background	3
1.1	IEC Inverse-Time Characteristics	3
1.2	Project Objectives	3
2	Hardware System	4
2.1	Pin Assignment	5
2.2	System Block Diagram	5
3	Signal Processing and Trip Algorithm	6
3.1	Current Measurement	6
3.2	Inverse-Time Logic	6
3.3	Program Excerpts	7
4	Expected Time–Current Behavior	8
4.1	Theoretical Characteristic	8
4.2	Trip Times with the Default Settings	9
4.3	Timing Accuracy of the Program	9
4.4	Test Procedure	9
5	Limitations and Possible Improvements	10
6	Conclusion	10

1 Introduction and Theoretical Background

Overcurrent protection is the most widely used form of protection in radial distribution systems. An overcurrent relay operates when the measured current exceeds a pickup value I_s . To obtain selectivity between a relay and the backup relay upstream of it, *inverse definite minimum time* (IDMT) characteristics are used: the larger the current, the shorter the operating time.

1.1 IEC Inverse-Time Characteristics

The IEC 60255-151 standard [1] defines the operating time of an inverse-time relay as

$$t = TMS \cdot \frac{\beta}{\left(\frac{I}{I_s}\right)^\alpha - 1}, \quad (1)$$

where t is the operating time in seconds, I the measured current, I_s the pickup current, TMS the time multiplier setting that shifts the curve vertically, and α, β the constants of the chosen curve. The ratio $PSM = I/I_s$ is called the plug setting multiplier.

Table 1: Standard IEC curve constants.

Characteristic	β	α
Standard (normal) inverse	0.14	0.02
Very inverse (implemented)	13.5	1
Extremely inverse	80	2
Long-time inverse	120	1

The **Very Inverse** curve ($\alpha = 1, \beta = 13.5$) is steeper than the standard inverse curve. It is suitable where the fault current falls rapidly with the distance from the source, because relays closer to the source can be given clearly longer operating times than those at the end of the feeder.

1.2 Project Objectives

- Measure a 50 Hz AC load current with a ZMCT103C and an Arduino ADC.
- Implement Eq. (1) with $\alpha = 1, \beta = 13.5$ in embedded C/C++.
- Trip a relay module after the calculated delay, ignoring disturbances that clear before the delay has elapsed.
- Provide local indication (LCD and LEDs) and a manual reset.

2 Hardware System

The prototype is built from low-cost, commercially available modules (Figure 1) and tested on a 12 V AC / 1 A bench.

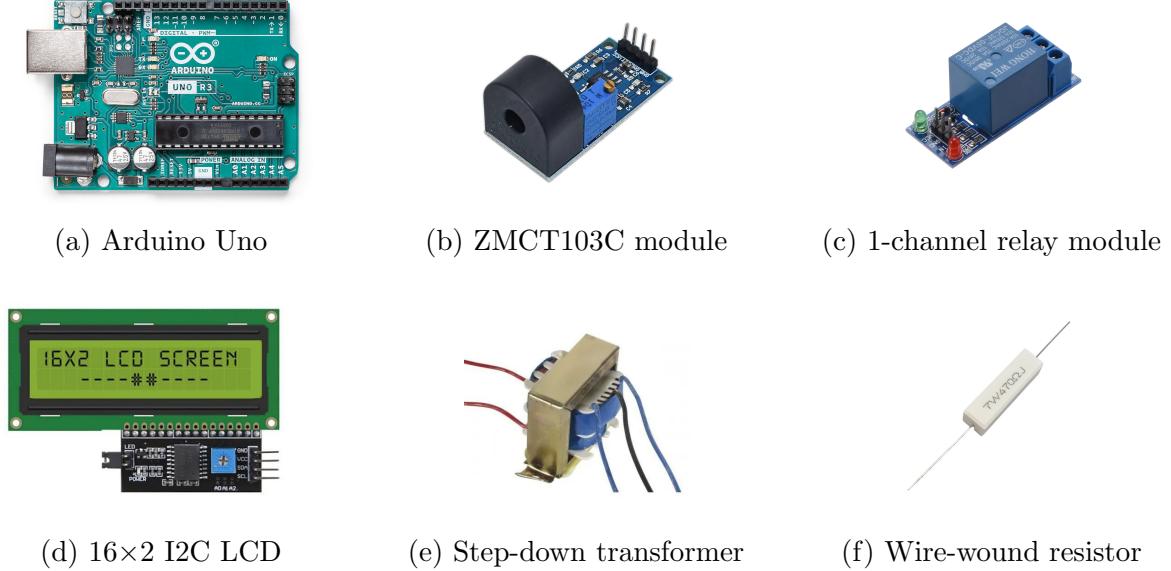


Figure 1: Main components of the prototype (product images shown for illustration).

- **Arduino Uno:** samples the sensor signal, evaluates Eq. (1), and controls the outputs.
- **ZMCT103C current transformer module:** galvanically isolates the measurement from the load circuit and delivers a small AC voltage proportional to the load current; the module output is biased around mid-supply so that the 0–5 V Arduino ADC can read both half-cycles.
- **1-channel 5 V relay module:** its contact is connected in series with the load and opens the circuit when the relay trips. The module used is *active-low*: the Arduino drives the input pin HIGH in normal operation and LOW to trip.
- **16×2 I2C LCD (address 0x27):** shows the measured current, the calculated trip time and the relay status.
- **Green and red LEDs:** green = normal, red = tripped.
- **Reset push button:** restores normal operation after a trip.
- **Step-down transformer and wire-wound resistors:** supply the 12 V AC test circuit and provide the adjustable load used to create overload conditions.

2.1 Pin Assignment

Table 2: Arduino Uno connections.

Pin	Function	Direction
A0	ZMCT103C output (analog)	Input
D8	Relay module input (LOW = trip)	Output
D7	Green LED (normal)	Output
D6	Red LED (tripped)	Output
D2	Reset push button (INPUT_PULLUP, pressed = LOW)	Input
A4/A5	I2C LCD (SDA/SCL)	Bus

2.2 System Block Diagram

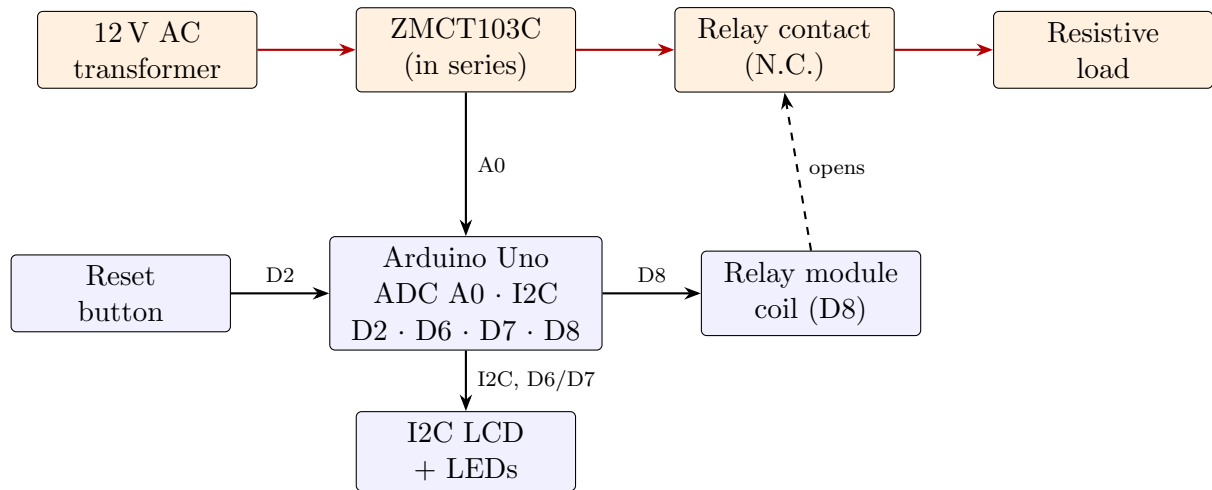


Figure 2: Block diagram: red arrows = load current path, black arrows = control and signal paths.

3 Signal Processing and Trip Algorithm

3.1 Current Measurement

The Arduino reads the ZMCT103C output with its 10-bit ADC (0–1023 counts for 0–5 V). In each measurement:

1. The ADC is sampled continuously for a window of 40 ms, which is two cycles of the 50 Hz waveform.
2. The maximum and minimum readings in the window are recorded.
3. The peak-to-peak voltage is $V_{pp} = \frac{ADC_{max} - ADC_{min}}{1023} \times 5 \text{ V}$.
4. For a sinusoid, $V_{rms} = \frac{V_{pp}/2}{\sqrt{2}} = \frac{V_{pp}}{2.8284}$.
5. The current is $I = K V_{rms}$ with the calibration factor $K = 5.0 \text{ A/V}$, determined against the test bench. Values below 0.08 A are treated as zero (noise dead-band).

Because the DC bias cancels in $ADC_{max} - ADC_{min}$, no offset removal is needed. Using the maximum and minimum is fast and needs no multiplication per sample, but it is an RMS *estimate* that is exact only for a sinusoidal current; distorted waveforms introduce an error (see Section 5).

One ADC count of peak-to-peak swing corresponds to $\frac{5/1023}{2.8284} \times 5 \approx 8.6 \text{ mA}$, which is the resolution of the current measurement.

3.2 Inverse-Time Logic

After each measurement the relay executes the following steps (see also Figure 3):

1. If $I \leq I_s$: the fault timer is cleared and the LCD shows **Status: NORMAL**.
2. If $I > I_s$: when the condition first appears, the start time of the fault is stored. The trip time t of Eq. (1) is calculated for the present current and shown on the LCD.
3. If the time elapsed since the start of the fault is greater than or equal to t , the relay trips.

If the current drops back to I_s or below before t has elapsed, the fault is considered transient and the timer restarts at the next pickup. This prevents nuisance trips on short overloads such as motor-starting inrush.

Trip action. On a trip the Arduino drives the relay input LOW (opening the load circuit), turns the green LED off and the red LED on, and shows **SYSTEM TRIPPED! Press to Reset**. The relay stays latched, and the measurement stops, until the reset button is pressed.

Default settings. $I_s = 0.4 \text{ A}$, $TMS = 0.05$, $\alpha = 1$, $\beta = 13.5$ (chosen for the 12 V AC / 1 A bench). Substituting in Eq. (1) gives

$$t = \frac{13.5 \times 0.05}{I/0.4 - 1} = \frac{0.675}{I/0.4 - 1} \text{ s.} \quad (2)$$

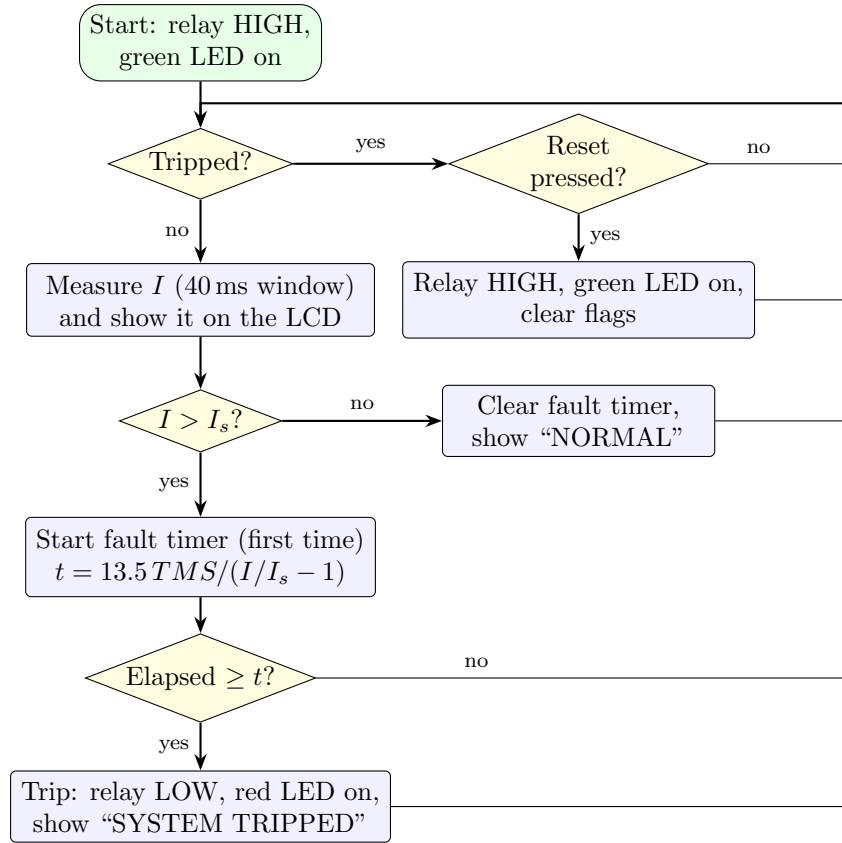


Figure 3: Flowchart of the relay program (one pass of the main loop; the loop repeats continuously, with a 50 ms pause at the end of each pass).

3.3 Program Excerpts

The two functions that implement the measurement and the trip decision (full code: Code/Protection_Proj.ino):

```

1 float readACCurrent() {
2     uint32_t samplePeriod = 40;           // 2 cycles at 50 Hz
3     uint32_t t_start = millis();
4     int max_val = 0, min_val = 1023;
5     while (millis() - t_start < samplePeriod) {
6         int adc_val = analogRead(SENSOR_PIN);
7         if (adc_val > max_val) max_val = adc_val;
8         if (adc_val < min_val) min_val = adc_val;
9     }
10    float v_swing = ((max_val - min_val) / 1023.0) * 5.0;
11    float v_rms = v_swing / 2.8284;
12    float current = v_rms * CALIBRATION_FACTOR; // 5.0
13    if (current < 0.08) current = 0.0;
14    return current;
15 }

```

```

1 if (current > Is) {
2   if (!faultActive) { faultActive = true; faultStartTime = millis();
3     }
4   float tripTimeSeconds = (BETA * TMS) / (pow((current / Is), ALPHA)
5     - 1.0);
6   unsigned long tripTimeMillis = tripTimeSeconds * 1000;
7   if ((millis() - faultStartTime) >= tripTimeMillis) triggerRelay();
8 } else {
9   faultActive = false;
10 }

```

4 Expected Time–Current Behavior

4.1 Theoretical Characteristic

Figure 4 plots Eq. (1) for the implemented curve ($\alpha = 1$, $\beta = 13.5$) for three TMS values; the default setting is $TMS = 0.05$. The time falls steeply just above the pickup value and flattens at high multiples, so mild overloads are tolerated for seconds while severe faults are cleared quickly.

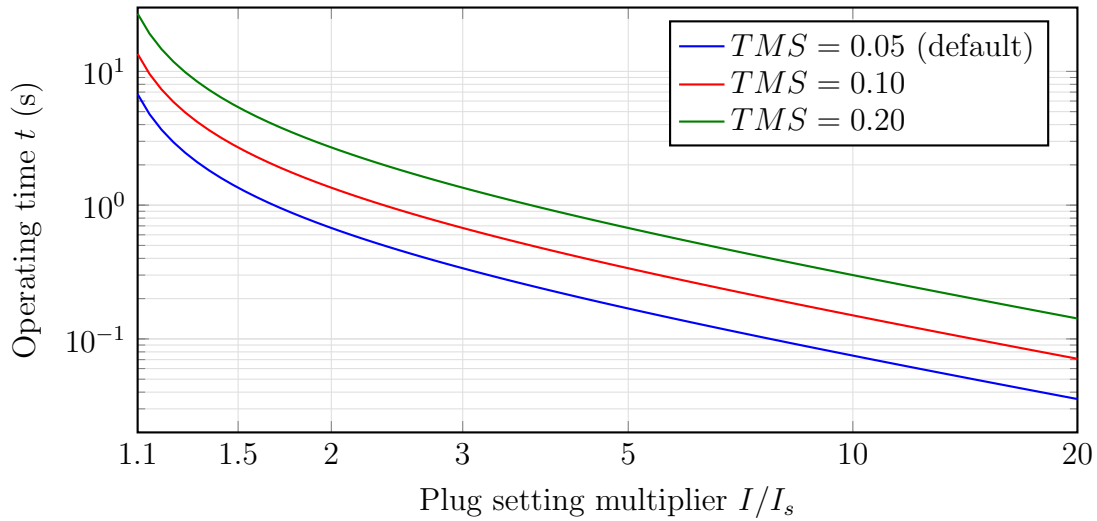


Figure 4: IEC Very Inverse characteristic implemented by the relay (calculated from Eq. (1)).

4.2 Trip Times with the Default Settings

For the 1 A bench the load current can reach about $PSM = 2.5$ with $I_s = 0.4$ A. Table 3 lists the calculated operating times in this range.

Table 3: Calculated operating time for $I_s = 0.4$ A and $TMS = 0.05$.

Load current (A)	I/I_s	Operating time (s)
0.5	1.25	2.70
0.6	1.50	1.35
0.7	1.75	0.90
0.8	2.00	0.675
0.9	2.25	0.54
1.0	2.50	0.45

4.3 Timing Accuracy of the Program

One pass of the main loop takes at least 40 ms (measurement) plus 50 ms (pause), and the LCD updates add a few more milliseconds. The elapsed time is therefore checked about every 0.1 s, so the trip is delayed by up to roughly one loop period beyond the calculated time. This is negligible for the operating times in Table 3 (0.45–2.7 s) but would matter for the very short times produced at high multiples of I_s .

4.4 Test Procedure

The relay is verified on the bench in the following way:

1. Set the load with the wire-wound resistors so that the current is below I_s ; the LCD must show **Status: NORMAL** and the green LED must stay on.
2. Increase the load to a chosen current above I_s (for example 0.6, 0.8 and 1.0 A) and measure the time from the moment of the overload to the opening of the relay contact with a stopwatch or by recording the LCD/serial output; compare it with Table 3.
3. Apply a short overload that is removed before the calculated time; the relay must not trip.
4. After a trip, press the reset button; the relay must close and the green LED must light.

5 Limitations and Possible Improvements

- **RMS estimation:** the peak-to-peak method is accurate only for sinusoidal currents. A true RMS calculation (sampling the waveform, removing the DC offset, squaring, averaging over whole cycles and taking the square root) would be more accurate for distorted currents.
- **Time integration:** the relay compares the elapsed time with the trip time re-computed for the present current. A numerical relay normally integrates the ratio $\Delta t/t(I)$ over time, which handles a changing fault current more accurately.
- **Timing resolution:** the loop period of about 0.1 s limits the accuracy at high multiples of I_s ; a hardware timer or interrupt-driven sampling would give finer timing.
- **Fixed settings:** I_s and TMS are constants in the code. Push buttons and EEPROM storage would allow field adjustment.
- **Scope:** the prototype protects a single-phase, low-voltage bench circuit. It has no instantaneous (high-set) element, no earth-fault element, and it is an educational model, not a certified protection device.
- **Other characteristics:** the constants α, β can be replaced with those of Table 1 to implement the standard, extremely or long-time inverse curves.

6 Conclusion

A microcontroller-based overcurrent relay with the IEC Very Inverse characteristic was designed and built using an Arduino Uno, a ZMCT103C current transformer module and a relay module. The relay estimates the load current, calculates the operating time from I_s and TMS , trips and latches when a fault lasts long enough, ignores disturbances that clear earlier, and provides local indication and a manual reset. The report documented the hardware connections, the measurement method with its resolution, the trip algorithm, the expected time-current behavior, and the main limitations. The project connects the theory of inverse-time protection with a low-cost embedded implementation and gives a base for extensions such as true RMS measurement, integrated timing and adjustable settings.

References

- [1] IEC 60255-151, *Measuring relays and protection equipment – Part 151: Functional requirements for over/under current protection*, International Electrotechnical Commission.
- [2] Arduino documentation, *analogRead()* and *millis()*, <https://www.arduino.cc/reference/en/>.